# Railway Reservation Simulator with Disruption Refund Advisor

Software Requirements Specification · Revision 1.0 · 24 October 2026

Soham More · Department of Computer Science & Engineering, Manipal University Jaipur · CSC2104 (Java OOP) and CSC2131 (RDBMS Lab)

Status: implementation baseline once section 15 is signed off. Nothing in this document is implemented or tested. It supersedes v0.1, v0.2 and v0.3.

## Revision summary (v0.2 to v1.0)

- Delivery window is now 3 to 4 days from 24 October 2026. Scope is unchanged; section 12 is rebuilt around parallel tracks and daily gates.
- Linked missed-connection refunds are deferred, not closed. The general IRCTC refund rules list them (linked PNRs, TDR within three hours of the actual arrival of the train travelled), while the separate IRCTC-trains document excludes connecting tickets on IRCTC-operated trains.
- Decision order rebuilt: decisive negatives first, then missing or unverified facts, then the deadline. A 100-minute delay with no filing attempt is NOT_ELIGIBLE, not UNRESOLVED.
- Prospective mode added: a train already more than three hours late that has not yet departed gives ELIGIBLE with an open deadline and the action file before actual departure.
- Event integrity rules added: an evaluation uses only events recorded at or before its evaluation time, and a delay that does not equal actual minus scheduled departure gives EVENT_CONFLICT.
- INV-05 gets a database-level final guard (SEAT_ALLOCATION carries journey and passenger under a unique key); INV-08 checks those copies against the booking.
- Parity is file-based: each engine writes canonical output files and a script compares them. No JDBC and no Tier 3 are needed.
- Fixture files use a plain line format readable with the Java standard library. D-05 is split; D-15 to D-17 and R-11 to R-15 are added.
- Source labels, warnings and action codes are defined once (section 5).

### Changes in the final 1.0 revision

- Added section 16, future scope.
- Dropped REFUND_RULE_FAMILY and the rule-publication procedure (15 tables). Overlapping rule intervals are now rejected by a trigger; concurrent-safe publication moved to future scope.
- The RefundPolicy hierarchy and its direct tests now start on day 1 (track A1); decision order, event integrity and prospective mode stay in A3.

# 1. Introduction

## 1.1 Purpose

This SRS is the implementation baseline for a teaching simulator of a railway reservation system with a rule-driven disruption refund advisor. It is the single source of truth for code, tests, report and viva. Tier 1 (Java) and Tier 2 (MySQL) are implemented independently and must agree on every shared fixture.

## 1.2 Product summary

The reservation engine models trains, dated journeys, seat classes, bookings with unique PNRs, cancellation, waitlists and promotion. It shows how concurrent requests over-allocate the last seat in an unsafe implementation, and how the corrected Java monitor design and the lock-first MySQL transaction design prevent it.

The refund advisor evaluates a disruption case from explicit, source-labelled events, an effective-dated sourced rule and an explicit evaluation time. It returns an outcome, reason codes, the rule version, a deadline or deadline basis, a required action and a simulated refund amount. Uncertainty never becomes a negative verdict: missing, unverified or conflicting facts give UNRESOLVED.

## 1.3 Scope

### In scope

- Reservation core: passengers, trains, journeys, classes, seats, bookings, PNRs, cancellation, waitlist, promotion, audit.
- Concurrency: Java threads with monitor synchronization and a bounded FIFO queue; MySQL transactions, lock-first booking, savepoints and two-session experiments.
- Refund advisor for two rule families (train over three hours late with the passenger not travelling; train cancelled) on confirmed simulated e-tickets, in retrospective and prospective modes.
- Rules as data: effective-dated, sourced, verification-flagged, protected from overlapping intervals by a trigger.
- Deterministic cases: events, evaluations, normalized reason and event links, plain-text summary.
- Two independent engines, shared fixtures and a file-based parity check.

### Out of scope

- Live train status, live PNR data, real inventory, payments, IRCTC or RailMadad integration. Collecting credentials or real passenger data. Filing a real TDR.
- RAC, waitlisted, partially confirmed, party or family PNR, Tatkal and i-ticket refund adjudication. The advisor refuses these with UNSUPPORTED_TICKET_SCOPE. The official rules do cover some of them, so this is a project scope choice, not a rule statement.

- Linked missed-connection refunds (deferred, section 5.2), partial cancellation, diversion, AC failure, coach problems, machine learning, production hosting.
- Legal or financial advice. Every advisor output is labelled a simulation.

### 1.4 Definitions

| Term | Meaning |
|---|---|
| PNR | Unique simulated booking reference, never reused. |
| Journey | One train on one date for a defined boarding-to-destination segment. |
| Journey class | One seat class (Sleeper, AC, General) of a journey. Secondary lock anchor. |
| Group reference | Shared identifier for several individual bookings; each keeps its own PNR. |
| Active allocation | The current seat assignment of a CONFIRMED booking. |
| Waitlist entry | Queue record of a WAITLISTED booking, ordered by booked_at then booking_id. |
| Disruption case | A record linked to one supported booking and one disruption reason. |
| Case event | A timestamped fact with event time, recorded time, type, source label and optional value. |
| Rule version | A dated, sourced refund-policy row with a verification flag. |
| Evaluation | An immutable result computed from a fixed rule version, an event set and an evaluation time. |
| Policy applicability time | The instant used to choose the rule version. Defaults to the scheduled departure at the boarding station. |
| TDR | Ticket Deposit Receipt, the online refund claim for certain cases. The simulator never files one. |
| Parity | Java and MySQL produce identical canonical output for the same fixture. |

## 2. Overall description

### 2.1 Architecture tiers

| Tier | Component | Independence |
|---|---|---|
| 1 | Standalone Java console simulator: domain, agents, queue, safe and unsafe races, refund policies, file audit | No database, JDBC or network |
| 2 | MySQL 8 schema, constraints, views, functions, procedures, triggers, cursor, audit, savepoints, two-session experiments | Runs without Java |
| Parity | Fixture files, canonical output files, comparison script | Reads files only; no JDBC |
| 3 (optional) | JDBC adapter, local HTTP API, frontend | Not planned; only if every gate passes with time left |

### 2.2 Users

| User | Main use |
|---|---|
| Booking clerk (simulated) | Create passengers, book, cancel, look up PNR, view availability and waitlist |
| Simulated passenger | Open a disruption case, add events, evaluate, read the summary |
| Evaluator | Run scenarios, SQL scripts, races, fixtures and parity; conduct the viva |
| Rule reviewer | Add an effective-dated rule as a row with its source metadata; overlaps are rejected by a trigger |

### 2.3 Operating environment and constraints

- Java: course-approved JDK (17 LTS if unspecified), console only, plain `javac` and scripts. No third-party libraries in the core. Tests use a self-contained runner class.
- **Java style default.** Until the official CSC2104 handout is checked, write the queue and agents in the most restrictive style: fixed-size arrays, `synchronized`, `wait`, `notifyAll`, `Thread` and `Runnable`, no `java.util.concurrent`, no collection classes in core code. That satisfies a strict handout and a permissive one. Collections and streams are allowed only in test and fixture-loading code unless the handout forbids them there too.
- Database: MySQL 8.0.16 or later, InnoDB, MySQL syntax only. Where a lab topic is taught with Oracle PL/SQL terms, document the MySQL equivalent. Do not paste Oracle-only MERGE or INSERT ALL.

- No network is needed at run time. Rule data is seeded; policy research is a build-time activity.
- All journey timestamps are Asia/Kolkata. Fixtures carry explicit offsets. Evaluation never reads the machine clock.
- Tooling scripts (fixture conversion, parity diff) may use Python 3 or shell. They are test tooling, not assessed engine code.

### 2.4 Assumptions

- Demo scale: a few trains, tens of seats per class, synthetic data only. One booking holds one seat.
- The advisor accepts only CONFIRMED bookings with ticket_type SIMULATED_E_TICKET.
- A rule row with verified = false can never produce ELIGIBLE or NOT_ELIGIBLE.
- The team confirms the exact deadline and whether MySQL is accepted for the assessed RDBMS portion (section 15).

## 3. Reservation domain rules and invariants

### 3.1 Booking lifecycle

| Current status | Allowed next | Notes |
|---|---|---|
| (new) | PENDING | Creation is audited |
| PENDING | CONFIRMED, WAITLISTED, CANCELLED | Allocation or queueing |
| CONFIRMED | CANCELLED | Releases one allocation, attempts one promotion |
| WAITLISTED | CONFIRMED, CANCELLED | Promotion only; cancel frees no seat |
| CANCELLED | none | Terminal. A new request gets a new booking and PNR |

### 3.2 Business rules

- BR-01 A PNR is unique and never reused.
- BR-02 One booking holds one seat. A group is several bookings sharing a group reference.
- BR-03 A passenger holds at most one CONFIRMED booking per journey across all classes. Waitlisted fallbacks in other classes may remain.
- BR-04 Waitlist order is earliest booked_at, then booking_id.
- BR-05 A full class waitlists a valid request. This is a normal result, not an exception.

- BR-06 CANCELLED is terminal.
- BR-07 Cancelling a CONFIRMED booking releases exactly one seat and makes at most one promotion attempt in that class. Cancelling a WAITLISTED booking frees nothing.
- BR-08 Fares are non-negative and depend on class.
- BR-09 Promotion skips a waiter who already holds a CONFIRMED booking on the same journey in another class, without mutating that entry.
- BR-10 Booking, allocation and waitlist state change only through the Java service or the approved MySQL procedures.

### 3.3 Invariants

| ID | Invariant | Check |
|---|---|---|
| INV-01 | Confirmed bookings never exceed seats in the class | Count by class |
| INV-02 | A seat has at most one active allocation; a booking has at most one | UNIQUE keys plus query |
| INV-03 | Every CONFIRMED booking has exactly one allocation; no other status has one | Left-join query returns zero rows |
| INV-04 | Every WAITLISTED booking has exactly one waitlist entry; no other status has one | Left-join query returns zero rows |
| INV-05 | At most one CONFIRMED booking per passenger per journey | Journey-level lock plus UNIQUE(passenger_id, journey_id) on SEAT_ALLOCATION |
| INV-06 | Every committed creation or transition has an audit row; rolled-back work leaves none | Compare lifecycle with BOOKING_AUDIT |
| INV-07 | Promotion picks the earliest eligible waiter, at most one per cancellation | Deterministic fixture |
| INV-08 | journey_id and passenger_id on SEAT_ALLOCATION equal those of its booking and class | Join query returns zero mismatches |

### 3.4 Lock order

Locking only JOURNEY_CLASS cannot protect BR-03, because two requests by one passenger can lock different classes. A JOURNEY row is therefore the first lock; the class row is second. Booking, cancellation and promotion all use this order.

1. Begin a transaction. The first data-access statement is `SELECT ... FOR UPDATE` on the JOURNEY row. No plain availability or count read happens first.
2. Lock the JOURNEY_CLASS row. If several rows are needed, lock them in ascending key order. Never lock a class before its journey.
3. Validate the booking, the passenger's journey-level confirmed status and current availability, then change booking, allocation or waitlist rows.
4. Write audit rows in the same transaction and commit. On any error roll back everything. Deadlocks and lock timeouts are reported as retryable failures; a retry must not duplicate the logical booking.

Cancel by PNR needs the immutable journey and class identifiers. The procedure receives them with the PNR (or a pre-lock lookup of immutable columns only), locks JOURNEY then JOURNEY_CLASS, then verifies the booking belongs to them. A mismatch is rejected without mutation. The lookup never decides availability.

Java mirrors this: one shared journey monitor guards passenger-journey validation, seat claim, release and promotion. Per-class monitors alone are not sufficient.

## 4. Functional requirements: reservation core

Priority: M must. Every requirement here is in scope; section 12 gives the build order.

| ID | Requirement | Pri |
|---|---|---|
| FR-R01 | Model TRAIN, JOURNEY, JOURNEY_CLASS and fixed JOURNEY_SEAT inventory: at least two trains, all three classes, plus a zero-seat and a one-seat class in fixtures | M |
| FR-R02 | Create a PENDING booking with a generated unique PNR, never reused, including after cancellation | M |
| FR-R03 | One booking allocates one seat; group members are separate bookings with a shared group reference | M |
| FR-R04 | Confirm when inventory exists, otherwise WAITLISTED | M |
| FR-R05 | Enforce at most one CONFIRMED booking per passenger per journey across classes (BR-03, INV-05) | M |
| FR-R06 | List the waitlist in booked_at, booking_id order | M |
| FR-R07 | Cancel CONFIRMED: remove the allocation, set CANCELLED, attempt one earliest eligible promotion. Cancel WAITLISTED: remove the entry, set CANCELLED, release no seat | M |
| FR-R08 | Enforce the lifecycle in 3.1; illegal transitions raise ILLEGAL_TRANSITION and leave no partial change | M |
| FR-R09 | Fares by class use an abstract SeatClass with overriding Sleeper, AC and General subclasses | M |
| FR-R10 | Console: availability, passenger create/list, book, group book, cancel by PNR, PNR lookup, waitlist, race demo, open case, add event, evaluate, print summary, exit. Bad input returns to the menu | M |
| FR-R11 | Audit every creation and transition. Java: synchronized append-only file log. MySQL: triggers write BOOKING_AUDIT (AFTER INSERT and AFTER UPDATE) | M |
| FR-R12 | A BookingBackend interface; the console and agents depend on it | M |
| FR-R13 | Bounded FIFO queue per the Java style default (fixed array, wait/notifyAll). Shutdown rejects new work, drains the queue, wakes waiters and joins workers | M |
| FR-R14 | At least three booking-agent threads; correctness never depends on priorities or timing | M |

| ID | Requirement | Pri |
|---|---|---|
| FR-R15 | Unsafe check-then-claim experiment on a test-only inventory with a start gate that makes two workers see the same last seat | M |
| FR-R16 | Corrected inventory under the same workload with 10 workers over at least 100 repetitions never exceeds capacity | M |
| FR-R17 | Cancellation and promotion follow the lock order and BR-09; an ineligible first waiter is skipped without mutation | M |
| FR-R18 | Group booking: one transaction with a savepoint per member (at least three, all on one journey); a failing member rolls back to its own savepoint and the others commit once | M |
| FR-R19 | A test checklist records fixture ID, setup, expected, observed and evidence for every run | M |
| FR-R20 | Java cross-class safety: two concurrent requests by one passenger in different classes of one journey yield exactly one CONFIRMED | M |

## 5. Functional requirements: disruption refund advisor

### 5.1 Supported rule families

| Family | Behaviour | Source |
|---|---|---|
| LATE_OVER_3_HOURS_E_TICKET | Full modelled fare when the delay at the boarding station is strictly more than 180 minutes, the passenger did not travel, and the TDR filing attempt is strictly before actual departure | IRCTC Refund Rules clause 6(a); IRCTC-trains rules clause 7 |
| TRAIN_CANCELLED_E_TICKET | Full modelled fare, refunded automatically; no TDR required | IRCTC Refund Rules clause 5(a) |

The official text says the Zonal Railway decides TDR claims. The simulator never presents its result as a railway decision.

### 5.2 Deferred and excluded

- **Linked missed connection (deferred).** The general rules list it: PNRs must be linked and the TDR filed within three hours of the actual arrival of the train travelled. The IRCTC-trains document says connecting tickets are not allowed on IRCTC-operated trains. Both stand; they cover different train categories. Implement only after all gates pass and the rule scope is reviewed. Do not invent a connection-gap threshold.

- Excluded: partial cancellation, diversion, short termination, AC failure (the sources give 20 hours of actual arrival), coach issues, party and family tickets.
- The delay is measured at the passenger's boarding station (stated in the IRCTC-trains rules; the general rules say station commencing journey). Record this reading as an assumption in each rule row.
- Refund amount is the modelled base fare. Convenience fees and payment charges are not modelled.

### 5.3 Rule, event and source requirements

| ID | Requirement | Pri |
|---|---|---|
| FR-D01 | REFUND_RULE rows hold: family, version, effective_from, effective_to (NULL = open), threshold minutes, deadline basis, refund basis, ticket scope, source URL, source title and clause, source_checked_on, reviewer, verified flag. Intervals are [effective_from, effective_to); overlaps are rejected | M |
| FR-D02 | A rule cannot be marked verified without source URL, clause, checked date and reviewer. A used version is never edited; a change adds a version | M |
| FR-D03 | A BEFORE INSERT trigger on REFUND_RULE rejects an overlapping interval for the same family. Rule maintenance is a single-operator action; concurrent-safe publication is future scope (section 16) | M |
| FR-D04 | open_case accepts only CONFIRMED SIMULATED_E_TICKET bookings whose reason matches a supported family; anything else is rejected with UNSUPPORTED_TICKET_SCOPE and creates no case | M |
| FR-D05 | Case events store event_time, recorded_at, type, source label and optional value | M |
| FR-D06 | Evaluation takes the case, policy_applicability_time and evaluation_time as explicit inputs and never reads the clock | M |
| FR-D07 | An evaluation uses only events with recorded_at at or before evaluation_time. Later events are ignored and add the warning EVENTS_AFTER_EVALUATION_IGNORED | M |
| FR-D08 | Rule selection uses policy_applicability_time against the effective interval, never the latest inserted row | M |
| FR-D09 | Each evaluation is an immutable snapshot persisted with its rule version, outcome, reason and warning rows, deadline, action, amount and the exact event IDs used. New events create a new evaluation | M |
| FR-D10 | Output is a deterministic plain-text summary plus the canonical machine-readable record | M |
| FR-D11 | Every output carries the mandatory disclaimer (5.9) | M |

Event types and the sources accepted for a decisive conclusion:

| Event type | Decisive source | Notes |
|---|---|---|
| SCHEDULED_DEPARTURE | SIMULATED_OFFICIAL | Normally taken from the journey |
| DELAY_AT_BOARDING_MINUTES | SIMULATED_OFFICIAL | Latest official running-late reading as of recorded_at |
| ACTUAL_DEPARTURE | SIMULATED_OFFICIAL | The deadline cutoff for late running |
| TRAIN_CANCELLED | SIMULATED_OFFICIAL | |
| PASSENGER_TRAVELLED, PASSENGER_NOT_TRAVELLED | SIMULATED_OFFICIAL or PASSENGER_REPORTED | Self-reported adds warning PASSENGER_STATUS_SELF_REPORTED; it cannot be verified in the simulation |
| FILING_ATTEMPT | SIMULATED_INPUT | The simulated TDR filing time |

SIMULATED_OFFICIAL means a trusted fixture value, not a query to Indian Railways. A passenger-reported delay, departure or cancellation is never decisive.

### 5.4 Event integrity

- Two SIMULATED_OFFICIAL events of the same type with different values give EVENT_CONFLICT.
- When ACTUAL_DEPARTURE is present, DELAY_AT_BOARDING_MINUTES must equal the minutes from scheduled to actual departure, otherwise EVENT_CONFLICT.
- PASSENGER_TRAVELLED together with PASSENGER_NOT_TRAVELLED gives EVENT_CONFLICT.

### 5.5 Outcomes

| Outcome | Exact condition |
|---|---|
| ELIGIBLE | A verified rule and complete trusted facts meet every condition, with a deadline or an open deadline basis |
| NOT_ELIGIBLE | A verified rule and trusted facts show a condition is not met (delay at or under the threshold, or the passenger travelled) |
| DEADLINE_EXPIRED | Other conditions are met and an explicit FILING_ATTEMPT is at or after actual departure |
| UNRESOLVED | No rule, unverified rule, missing or unverified fact, or conflicting events. Never guessed |

### 5.6 Decision order

Late-running family:

1. Check the case reason matches the rule family. Select the rule by policy_applicability_time; none gives UNRESOLVED with RULE_NOT_FOUND.
2. Selected rule not verified: UNRESOLVED with RULE_SOURCE_UNVERIFIED, whatever the numbers say.
3. Apply the integrity rules in 5.4. Any conflict: UNRESOLVED with EVENT_CONFLICT.
4. **Decisive negatives first.** If an official delay is at or under the threshold, add DELAY_NOT_OVER_THRESHOLD. If the passenger travelled, add PASSENGER_TRAVELLED. Any decisive negative: NOT_ELIGIBLE with all that apply. Filing facts are not needed.
5. **Missing or unverified facts.** No delay fact: DELAY_UNKNOWN. Only a passenger-reported delay: DELAY_UNVERIFIED. No travel fact: TRAVEL_STATUS_UNKNOWN. Any of these: UNRESOLVED with all that apply.
6. **Deadline.** Delay is over the threshold and the passenger did not travel.
    - ACTUAL_DEPARTURE recorded: no FILING_ATTEMPT gives UNRESOLVED with FILING_ATTEMPT_UNKNOWN (evaluation time alone never proves late filing). Filing strictly before departure gives ELIGIBLE with FILED_BEFORE_DEPARTURE. Filing at or after departure gives DEADLINE_EXPIRED with FILED_AFTER_DEPARTURE.
    - ACTUAL_DEPARTURE not yet recorded (prospective mode): ELIGIBLE with DELAY_OVER_THRESHOLD and ACTUAL_DEPARTURE_PENDING, an open deadline, and action FILE_TDR_BEFORE_ACTUAL_DEPARTURE. If a FILING_ATTEMPT is already recorded, the action is FILED_AWAITING_DEPARTURE_CONFIRMATION.
7. Persist the snapshot, reason and warning rows and event links in one transaction, then render the summary.

Train-cancelled family: steps 1 to 3 as above. An official TRAIN_CANCELLED event gives ELIGIBLE with TRAIN_CANCELLED_AUTO_REFUND, action NO_TDR_REQUIRED and the full modelled fare. Only a passenger-reported cancellation gives UNRESOLVED with TRAIN_CANCELLATION_UNVERIFIED. None gives UNRESOLVED with TRAIN_CANCELLATION_UNKNOWN.

### 5.7 Reason, warning and action codes

| Area | Codes |
|---|---|
| Rule and scope | RULE_NOT_FOUND, RULE_SOURCE_UNVERIFIED, RULE_INTERVAL_CONFLICT, UNSUPPORTED_TICKET_SCOPE |
| Delay | DELAY_OVER_THRESHOLD, DELAY_NOT_OVER_THRESHOLD, DELAY_UNKNOWN, DELAY_UNVERIFIED, ACTUAL_DEPARTURE_PENDING |
| Travel and deadline | PASSENGER_TRAVELLED, TRAVEL_STATUS_UNKNOWN, FILING_ATTEMPT_UNKNOWN, FILED_BEFORE_DEPARTURE, FILED_AFTER_DEPARTURE |
| Cancellation | TRAIN_CANCELLED_AUTO_REFUND, TRAIN_CANCELLATION_UNKNOWN, TRAIN_CANCELLATION_UNVERIFIED |
| Integrity | EVENT_CONFLICT, EVALUATION_FAILED |
| Reservation | ILLEGAL_TRANSITION, CAPACITY_FULL_WAITLISTED, NO_ELIGIBLE_WAITER, PASSENGER_ALREADY_CONFIRMED, ALLOCATION_MISMATCH |
| Parity | PARITY_MATCH, PARITY_MISMATCH |
| Warnings | PASSENGER_STATUS_SELF_REPORTED, EVENTS_AFTER_EVALUATION_IGNORED |
| Actions | FILE_TDR_BEFORE_ACTUAL_DEPARTURE, FILED_AWAITING_DEPARTURE_CONFIRMATION, MODELLED_TDR_FILED_BEFORE_DEPARTURE, NO_TDR_REQUIRED, NONE_DEADLINE_PASSED, NONE |

Display order is deterministic: primary outcome reason first, then supporting codes in canonical alphabetical order. Parity compares sorted sets.

### 5.8 Seed rules

| Family | Version | Threshold | Deadline basis | Refund basis | verified |
|---|---|---|---|---|---|
| LATE_OVER_3_HOURS_E_TICKET | 1 | 180 minutes, strictly greater | ACTUAL_DEPARTURE, strictly before | FULL_FARE | false until Appendix C is complete |
| TRAIN_CANCELLED_E_TICKET | 1 | not applicable | AUTOMATIC | FULL_FARE | false until Appendix C is complete |

While verified is false the advisor returns UNRESOLVED with RULE_SOURCE_UNVERIFIED. That is itself a demonstration case. Test fixtures may carry their own verified synthetic rule rows.

### 5.9 Mandatory disclaimer

"Simulated guidance based on the rule version shown. This is not an official refund decision. This system does not file a TDR or guarantee a refund."

## 6. Data model and integrity

### 6.1 Tables (15)

| Table | Purpose | Key constraints |
|---|---|---|
| PASSENGER | Synthetic passenger | PK passenger_id; no real personal data |
| TRAIN | Simulated train | PK; UNIQUE train_number |
| JOURNEY | Train, service date, segment, scheduled departure. First lock anchor | UNIQUE(train_id, service_date, from_station, to_station) |
| JOURNEY_CLASS | One class of one journey with base fare. Second lock anchor | UNIQUE(journey_id, class_code); UNIQUE(journey_class_id, journey_id) for composite FKs; CHECK fare >= 0 |
| JOURNEY_SEAT | One physical simulated seat. Capacity is the count of these rows, never a stored total | UNIQUE(journey_class_id, seat_number); UNIQUE(seat_id, journey_class_id) |
| BOOKING | PNR, passenger, class, fare DECIMAL(10,2), status, booked_at, optional group_ref, ticket_type | UNIQUE(pnr); CHECK status and ticket_type domains; CHECK fare >= 0; PNR derived from the AUTO_INCREMENT id and bookings are never deleted |
| SEAT_ALLOCATION | Current seat of a CONFIRMED booking. Carries journey_id and passenger_id as a documented, justified denormalization | UNIQUE(booking_id); UNIQUE(seat_id); UNIQUE(passenger_id, journey_id); composite FKs to JOURNEY_SEAT(seat_id, journey_class_id) and JOURNEY_CLASS(journey_class_id, journey_id) |
| WAITLIST_ENTRY | Queue entry of a WAITLISTED booking | UNIQUE(booking_id); index (journey_class_id, created_at, booking_id) |
| BOOKING_AUDIT | Append-only creation and transition history | Updates and deletes blocked by trigger and by grants |

| Table | Purpose | Key constraints |
|---|---|---|
| REFUND_RULE | Effective-dated sourced rule version | UNIQUE(family_code, version); CHECK family_code is one of the supported families; CHECK that verified = true requires source URL, clause, checked date and reviewer; BEFORE INSERT trigger rejects overlapping intervals per family |
| DISRUPTION_CASE | Case on one supported booking and one family | FK booking_id; CHECK status domain |
| CASE_EVENT | Event with event_time, recorded_at, type, source label, optional value | CHECK on type and source label; FK case_id |
| CASE_EVALUATION | Immutable result: rule, applicability time, evaluation time, outcome, deadline or basis, action, amount, summary | Updates and deletes blocked by trigger |
| CASE_EVALUATION_CODE | One row per reason or warning code | PK(evaluation_id, code, code_kind); ordinal column for display order |
| CASE_EVALUATION_EVENT | Links an evaluation to every event it used, with the event's role | PK(evaluation_id, event_id) |

Document keys, functional dependencies and a 3NF argument for every table, including why SEAT_ALLOCATION carries journey_id and passenger_id and how INV-08 keeps those copies honest. No delimited code lists and no duplicated capacity totals.

### 6.2 Common data rules

- InnoDB everywhere. Money is DECIMAL(10,2) in MySQL and BigDecimal in Java; never float or double. Currency is INR, labelled as the simulator's modelled fare.
- Timestamps are DATETIME in Asia/Kolkata; event time (when the fact applies) is separate from recorded time (when it was entered).

- PK, FK, UNIQUE, NOT NULL and enforced CHECK constraints wherever applicable (MySQL 8.0.16 or later).
- Application accounts get no direct UPDATE or DELETE on BOOKING.status, SEAT_ALLOCATION or WAITLIST_ENTRY; state changes go through procedures.
- Rule intervals use [effective_from, effective_to) and may not overlap within a family. A BEFORE INSERT trigger rejects overlaps. Rule maintenance is a single-operator action; concurrent-safe publication is future scope (section 16).
- An evaluation and its code and event link rows are written in one transaction. Later events create a new evaluation.

### 6.3 Routines, functions and triggers

| Routine | Responsibility |
|---|---|
| book_seat | Transaction; first statement locks JOURNEY FOR UPDATE; lock JOURNEY_CLASS; validate passenger journey status; insert PENDING; allocate or waitlist; audit; commit; rollback on error |
| cancel_booking | Same lock order; verify the booking belongs to the given journey and class; cancel, remove allocation or entry, attempt one promotion, audit, commit |
| promote_waitlist | Cursor over the waitlist by booked_at, booking_id; skip a passenger already confirmed on the journey; promote the first eligible; never commits its caller |
| book_group | One transaction, shared group_ref, a savepoint per member; roll back to the failing member's savepoint only |
| occupancy_pct | DECIMAL(5,2) from 0.00 to 100.00 rounded to two places; counts confirmed allocations only; zero-seat class returns 0.00; uses COALESCE |
| open_case, add_case_event | Validate scope and event type; reject unsupported scope with UNSUPPORTED_TICKET_SCOPE |
| evaluate_case | Implement 5.6 exactly; persist snapshot, code rows and event links atomically |
| report_refresh | A cursor-based summary routine for a concrete course demonstration |
| Triggers | BEFORE INSERT forces initial status PENDING; BEFORE UPDATE rejects illegal transitions with SIGNAL SQLSTATE 45000; AFTER INSERT and AFTER UPDATE write BOOKING_AUDIT; block updates and deletes on audit and evaluation tables. A BEFORE INSERT trigger on REFUND_RULE rejects an overlapping interval for the same family. Triggers never COMMIT or ROLLBACK |

### 6.4 Transaction and concurrency rules

- Status changes and their audit rows commit or roll back together.

- Lock hierarchy: JOURNEY, then JOURNEY_CLASS, then booking, allocation and waitlist rows in ascending key order.
- No availability read before the JOURNEY lock.
- Transaction ownership belongs to the top-level procedure. Helpers do not commit.
- The unsafe experiment uses isolated demo tables with no protective keys and never touches the real schema.
- The corrected two-session run shows session 2 waiting on the JOURNEY row, then observing the committed result.

## 7. Interfaces and fixture contract

### 7.1 Java console

Menu actions: show availability; create and list passengers; book; group book; cancel by PNR; look up PNR with history; show waitlist; run concurrency demo (unsafe then safe, printing claims against capacity and run count); open case; add event; evaluate; print summary; exit with orderly shutdown.

### 7.2 MySQL scripts, in order

1. 00_reset_schema.sql: targets only the dedicated simulator schema; safe to rerun.
2. 01_create_tables.sql: 15 tables, keys, checks, indexes.
3. 02_seed_data.sql: deterministic passengers, journeys, classes, seats, bookings and seed rules (verified = false).
4. 03_queries_views.sql: lab SELECTs, joins, aggregates, subqueries, views including one updatable view.
5. 04_functions.sql: occupancy_pct and any calculation helper.
6. 05_triggers.sql.
7. 06_procedures.sql: all routines in 6.3.
8. 07_experiments.sql plus a two-session run guide: unsafe race, corrected locking, stale-snapshot comparison.
9. 08_verify_invariants.sql: zero rows on valid data; finds the seeded unsafe fault.
10. 09_privileges.sql: least-privilege GRANT and REVOKE with SHOW GRANTS.

### 7.3 Refund fixture file format

One file per fixture under fixtures/refund/, plain lines, readable with the Java standard library:

```text
```

```
id=D-01
family=LATE_OVER_3_HOURS_E_TICKET
rule=1|2026-01-01T00:00:00+05:30||180|ACTUAL_DEPARTURE|FULL_FARE|true
fare=1250.00
booking=CONFIRMED|SIMULATED_E_TICKET
policy_applicability=2026-11-03T08:00:00+05:30
evaluation=2026-11-03T12:00:00+05:30
event=SCHEDULED_DEPARTURE|2026-11-03T08:00:00+05:30|2026-11-03T07:00:00+05:30|SIMU
event=DELAY_AT_BOARDING_MINUTES|2026-11-03T08:00:00+05:30|2026-11-03T10:45:00+05:3
expect.outcome=ELIGIBLE
expect.reasons=DELAY_OVER_THRESHOLD,FILED_BEFORE_DEPARTURE
expect.warnings=PASSENGER_STATUS_SELF_REPORTED
expect.deadline_at=2026-11-03T11:30:00+05:30
expect.action=MODELLED_TDR_FILED_BEFORE_DEPARTURE
expect.amount=1250.00
```

The event line is type, event_time, recorded_at, source, value. Appendix A lists the complete D-01 file. Reservation scenarios under fixtures/reservation/ list one step per line (BOOK, CANCEL, GROUP) and an expected final state dump.

A tool script converts each fixture into SQL calls for the MySQL engine. Java reads the same file directly. The tool is test infrastructure, not an assessed engine.

### 7.4 Canonical output and parity

Each engine writes out/<engine>/<fixture-id>.out with fixed keys in a fixed order: outcome, reasons (sorted, comma-separated), warnings (sorted), rule_id_version, policy_applicability, deadline_at or basis, action, currency, amount. Reservation scenarios write a sorted state dump: PNR, status, class, seat or waitlist position.

The parity script compares out/java and out/mysql file by file and against the expect lines, printing PARITY_MATCH or PARITY_MISMATCH with the differing field. A match shows the engines agree with the fixture oracle. It does not prove the policy reading is correct.

## 8. Non-functional requirements

| ID | Requirement | Evidence |
|---|---|---|
| NFR-01 | Determinism: same fixture, rule version and evaluation time give identical canonical output | Repeat runs, diff outputs |
| NFR-02 | Independence: Tier 1 runs without a database or network; Tier 2 without Java | Run each from its README |
| NFR-03 | All invariants INV-01 to INV-08 hold after every committed operation | Verification SQL and Java assertions return zero violations |
| NFR-04 | Corrected booking never over-allocates under staged contention | 100 runs with 10 workers; record seed and result |
| NFR-05 | A failed operation leaves no partial booking, allocation, waitlist or audit state | Rollback and savepoint fixtures |
| NFR-06 | Every refund result names its rule version, events used, outcome, codes, warnings, deadline or basis, action and amount | Summary text and CASE_EVALUATION_EVENT links |
| NFR-07 | Honesty: unverified rules and missing facts stay UNRESOLVED; every output has the disclaimer | Negative fixtures and output assertions |
| NFR-08 | Privacy: synthetic data only, no credentials, no real PNRs | Repository and fixture audit |
| NFR-09 | Reproducibility: a second person can reset, seed and run everything from the README | Fresh-checkout walkthrough |
| NFR-10 | Least privilege for database accounts | GRANT, REVOKE and SHOW GRANTS output |
| NFR-11 | Evidence integrity: every claim of passing rests on a captured log of a real run, never on assertion | evidence/ folder and the claims-against-evidence table |

## 9. Verification and acceptance tests

### 9.1 Reservation fixtures

| ID | Scenario | Expected |
|---|---|---|
| R-01 | Two Java agents, last seat, unsafe inventory, start gate | Over-claim recorded: the race is reproducible |
| R-02 | Corrected Java service, 10 workers, 100 repetitions | Never above capacity; one confirmed for the last seat |
| R-03 | Two MySQL sessions book the last seat with the lock-first procedure | Session 2 waits on the JOURNEY lock, then cannot over-book |
| R-04 | Cancel a confirmed booking with two waiters | One seat released; earliest eligible waiter promoted once |
| R-05 | First waiter already confirmed in another class | Skipped unchanged; next eligible promoted |
| R-06 | Cancel a waitlisted booking; cancel with an empty waitlist | No seat released; empty list exits cleanly |
| R-07 | Every forbidden transition | Rejected with ILLEGAL_TRANSITION; no partial audit |
| R-08 | Three-member group, member two fails after a partial insert | Only that member's work undone; others commit once |
| R-09 | SQL error injected before commit | Whole operation rolls back including audit rows |
| R-10 | Invariant suite on valid seed and on the isolated unsafe demo | Zero violations; the unsafe demo is flagged |
| R-11 | Java: one passenger requests two classes of one journey concurrently | Exactly one CONFIRMED |
| R-12 | MySQL: same race in two sessions; also a direct insert that breaks UNIQUE(passenger_id, journey_id) | Second session waits then waitlists or is rejected; the direct insert is refused by the key |
| R-13 | Bounded queue: FIFO order, blocks when full, blocks when empty, shutdown drains and joins | All behaviours observed |

| ID | Scenario | Expected |
|---|---|---|
| R-14 | occupancy_pct over seeded classes including zero seats | 0.00 to 100.00, two decimals, confirmed only, 0.00 for zero seats |
| R-15 | Privileges: reader can SELECT views but not modify; writer can execute procedures but not UPDATE BOOKING directly | SHOW GRANTS captured; denied statements show errors |

### 9.2 Reservation scenario parity

S-01 fill to capacity then waitlist; S-02 cancellation promotion order; S-03 skip of an ineligible waiter; S-04 illegal transition rejected; S-05 group booking with a failing member. Both engines run the same scenario file and produce identical state dumps.

### 9.3 Refund fixtures

| ID | Condition | Expected |
|---|---|---|
| D-01 | Official delay 210 consistent with actual departure 11:30; not travelled (self-reported); filing 15 minutes before departure | ELIGIBLE; DELAY_OVER_THRESHOLD, FILED_BEFORE_DEPARTURE; warning PASSENGER_STATUS_SELF_REPORTED; full fare; MODELLED_TDR_FILED_BEFORE_DEPARTURE |
| D-02 | Official delay exactly 180 | NOT_ELIGIBLE; DELAY_NOT_OVER_THRESHOLD |
| D-03 | Official delay 179 | NOT_ELIGIBLE; DELAY_NOT_OVER_THRESHOLD |
| D-03b | Official delay 181, filing before departure | ELIGIBLE |
| D-04 | Delay 181 reported by the passenger only | UNRESOLVED; DELAY_UNVERIFIED |
| D-05a | No delay fact; travel fact present | UNRESOLVED; DELAY_UNKNOWN |
| D-05b | Official delay 210; no travel fact | UNRESOLVED; TRAVEL_STATUS_UNKNOWN |
| D-05c | Official delay 100; no travel fact; no filing | NOT_ELIGIBLE; DELAY_NOT_OVER_THRESHOLD (decisive negatives first) |
| D-06a | Filing one second before actual departure | ELIGIBLE; FILED_BEFORE_DEPARTURE |
| D-06b | Filing exactly at actual departure | DEADLINE_EXPIRED; FILED_AFTER_DEPARTURE |
| D-06c | Filing one second after actual departure | DEADLINE_EXPIRED; FILED_AFTER_DEPARTURE |
| D-06d | Actual departure recorded; no filing event | UNRESOLVED; FILING_ATTEMPT_UNKNOWN |

| ID | Condition | Expected |
|---|---|---|
| D-07 | Official passenger-travelled fact; no delay fact | NOT_ELIGIBLE; PASSENGER_TRAVELLED |
| D-08 | Official train cancellation on a confirmed simulated e-ticket | ELIGIBLE; TRAIN_CANCELLED_AUTO_REFUND; NO_TDR_REQUIRED; full fare |
| D-08b | Cancellation reported by the passenger only | UNRESOLVED; TRAIN_CANCELLATION_UNVERIFIED |
| D-08c | No cancellation fact | UNRESOLVED; TRAIN_CANCELLATION_UNKNOWN |
| D-09 | Two rule versions; applicability time falls in the older; evaluation is later | Older version used |
| D-10 | No rule effective at applicability time | UNRESOLVED; RULE_NOT_FOUND |
| D-11 | Selected rule has verified = false | UNRESOLVED; RULE_SOURCE_UNVERIFIED |
| D-12 | Waitlisted booking or another ticket type | open_case rejected; UNSUPPORTED_TICKET_SCOPE; no case created |
| D-13 | Every fixture run in both engines | PARITY_MATCH on all canonical fields |
| D-14a | Two different official delay values | UNRESOLVED; EVENT_CONFLICT |
| D-14b | Delay 210 but actual minus scheduled departure is 395 minutes | UNRESOLVED; EVENT_CONFLICT |
| D-14c | Travelled and not-travelled both recorded | UNRESOLVED; EVENT_CONFLICT |
| D-15 | Prospective: official delay 210, not travelled, no actual departure, no filing | ELIGIBLE; DELAY_OVER_THRESHOLD, ACTUAL_DEPARTURE_PENDING; FILE_TDR_BEFORE_ACTUAL_DEPARTURE; open deadline |

| ID | Condition | Expected |
|---|---|---|
| D-16 | Prospective with a filing attempt recorded | ELIGIBLE; FILED_AWAITING_DEPARTURE_CONFIRMATION |
| D-17 | Actual departure recorded after the evaluation time | Event ignored; warning EVENTS_AFTER_EVALUATION_IGNORED; same result as D-15 |

### 9.4 Evidence rules

Every pass is backed by a captured log in evidence/ naming the command, date and output. The README carries a claims-against-evidence table. A claim with no log is not made. Failures found and fixed are kept in the log, not hidden.

## 10. Course traceability

### 10.1 CSC2104 Java OOP (provisional until the official handout is checked)

| Concept | Where it appears |
|---|---|
| Classes, encapsulation, constructors | Passenger, Booking, JourneyClass, SeatInventory, DisruptionCase, CaseEvent, RefundRule; validated constructors; private state |
| Inheritance, abstract classes, overriding | Abstract SeatClass with Sleeper, AC, General; abstract RefundPolicy with LateRunning and TrainCancelled |
| Interfaces, polymorphism | BookingBackend; RefundPolicy and a RuleEvaluator contract; console depends on contracts |
| Exceptions | Illegal transition, duplicate confirmed booking, missing rule, unsupported scope, invalid input; custom exception classes |
| Arrays and references | Fixed-size seat inventory and the fixed-array FIFO queue |
| Threads, Runnable, synchronization | Three or more agents, bounded queue, journey monitor, start-gate races, shutdown and join |
| File I/O | Synchronized audit log, fixture loading, canonical output files |
| Copy behaviour, inner classes, other handout topics | Only where the handout requires them; otherwise a small separate exercise |

### 10.2 CSC2131 RDBMS Lab

| Lab topic | Project work |
|---|---|
| ER, keys, normalization | ER diagram for 15 tables; candidate keys, functional dependencies, 3NF argument, the SEAT_ALLOCATION denormalization |
| DDL and data types | InnoDB tables, DATETIME, DECIMAL, constrained domains, indexes |
| Integrity constraints | PK, FK, UNIQUE, NOT NULL, CHECK as in 6.1 |
| DML, SELECT, WHERE, ORDER BY | Seed, book, cancel, evaluate; filters by status, class, rule, date |
| Functions, aggregates, GROUP BY, HAVING | occupancy_pct; confirmed and waitlisted counts; average fare and refund summaries |
| Joins, subqueries | Inner, outer, self, cross; NATURAL JOIN shown only with a warning; correlated subqueries for conflicts |
| Views | Waitlist order view, latest evaluation per case view, one updatable view tested |
| Privileges (Lab 11) | Engine writer, read-only reader, administrator; GRANT, REVOKE, SHOW GRANTS |
| TCL (Lab 12) | COMMIT, ROLLBACK, SAVEPOINT in booking and group booking |
| Triggers, procedures, cursors (Lab 12) | Audit and transition triggers, routines in 6.3, cursor-based promotion and report refresh |

## 11. Deliverables and repository layout

```text
docs/SRS.md  docs/IMPLEMENTATION_PLAN.md  docs/TRACEABILITY.md  docs/VIVA_NOTES.md
java/src  java/test  java/build.sh  java/run.sh
mysql/00_reset_schema.sql ... mysql/09_privileges.sql  mysql/run_all.sh
fixtures/refund/  fixtures/reservation/
tools/fixture_to_sql, tools/parity, tools/run_all
out/java  out/mysql  evidence/  README.md  REPORT.md
```

Every deliverable is traced to a requirement in docs/TRACEABILITY.md: requirement ID, file, test or fixture, evidence log.

## 12. Plan (3 to 4 days)

The exact deadline is an open item. Days below assume the first build day is 25 October 2026 and a fourth day exists as buffer; if only three days exist, day 4 work folds into the end of day 3. Three tracks run in parallel from the start.

| Track | Work |
|---|---|
| A: Java | A1 domain, state machine, sequential service, audit log, and the RefundPolicy hierarchy with direct unit tests of the threshold and cancellation checks (teammate slice, starts day 1). A2 queue, agents, unsafe and safe races, cross-class safety. A3 decision order, event integrity, prospective mode, fixture runner, canonical output. A4 console and shutdown |
| B: MySQL | B1 schema, seed, queries, views. B2 triggers and functions. B3 lock-first procedures, savepoints, cursor. B4 experiments, invariants, privileges. B5 refund tables, the overlap trigger, open_case, evaluate_case |
| C: Parity and evidence | Fixtures in plain language then files; fixture_to_sql tool; parity script; evidence logs; README, report, ER diagram, viva notes |

| Day | Gate | Exit criteria |
|---|---|---|
| 1 (25 Oct) | Foundations | Repo, environment checks and docs/IMPLEMENTATION_PLAN.md done. All fixtures written. A1 (with the RefundPolicy hierarchy and its direct tests) and B1 and B2 done: sequential lifecycle passes in Java; schema loads and INV-01 to INV-08 return zero on seed; R-07 and S-01 to S-05 pass sequentially in both engines |
| 2 (26 Oct) | Concurrency and transactions | A2 and B3 and B4 done: R-01, R-02, R-11, R-13 in Java; R-03 to R-06, R-08, R-09, R-10, R-12, R-14, R-15 in MySQL; two-session experiment logs captured |
| 3 (27 Oct) | Refund advisor and parity | A3 and B5 done: D-01 to D-17 in both engines; PARITY_MATCH on every fixture and scenario; rules verified or deliberately left unverified with the demo case |
| 4 (28 Oct) | Evidence and viva | A4 done; README, report, ER diagrams, TRACEABILITY and VIVA_NOTES complete; fresh-checkout run reproduces; every member passes a mock viva; remaining regressions fixed |

A short teach-back each evening: every member explains the day's work aloud, and the agent updates docs/VIVA_NOTES.md from what was actually built.

If a gate is missed, this order is a safety valve, not a plan: report polish and optional query extras go first. Never cut the concurrency proof, transaction correctness, Lab 11 and 12 content, the

two rule families, or parity on D-01 to D-11. Linked connections and Tier 3 are not planned.

### 12.1 Suggested ownership

| Area | Owner |
|---|---|
| Passenger, Booking, SeatClass hierarchy, exceptions, audit log, RefundPolicy subclasses; MySQL seed data, SELECTs, views, privileges | Teammate with inheritance-level Java, with support |
| Journey monitor, queue, races; MySQL lock-first procedures, savepoints, triggers, evaluate_case; parity | Project lead |
| Plain-language fixtures, rule-source records, SQL run evidence, README, report, demo script | Other members |

Every member explains the whole project in 30 seconds and defends their own slice in depth. Commits go under each member's own name. Code written with an agent must be read, run and explained by the member who owns it before it counts as theirs.

## 13. Risks

| Risk | Mitigation |
|---|---|
| Full scope in 3 to 4 days | Parallel tracks, daily gates, fixtures written first, agent-driven build with strict evidence; the cut order above is a safety valve only |
| Viva on code the team did not write by hand | Daily teach-back, VIVA_NOTES per module, owners run and explain their slice |
| Handout restricts Java constructs | Default to the most restrictive style so either handout is satisfied |
| MySQL not accepted or not installed | Confirm with faculty; use a local container if needed; document PL/SQL equivalents |
| Refund rules amended or misread | Rules are data with source and date; saved dated PDF copies; verified flag; two-person review; disclaimer on all output |
| Java and MySQL disagree on edge cases | Boundary fixtures D-02, D-03, D-06a to D-06c; file-based parity from day 1 |
| Race tests flaky | Start gates, not timing; record seed and run count; two-session runs scripted with the mysql client |
| Evidence claimed without a real run | NFR-11; claims-against-evidence table; no pass without a log |
| Presented as a real passenger service | State plainly that it simulates published rules and files nothing |

## 14. References


1. IRCTC, Refund Rules and TDR Filing: https://contents.irctc.co.in/en/RefundCancellationRules.pdf, retrieved 24 October 2026. Clauses 5(a) and 6 and the table row for a missed connecting train. This was a live fetch; confirm it is the current amendment and save a dated copy under docs/sources/.
2. IRCTC, Cancellation and Refund Rules for IRCTC Trains: https://contents.irctc.co.in/en/CancellationRulesforIRCTCTrain.pdf, retrieved 24 October 2026. The late-running clause (cited above as clause 7; confirm the number), the connecting-ticket exclusion and the 20-hour AC-failure deadline.
3. IRCTC, E-Ticket Cancel guidance: https://contents.irctc.co.in/en/eticketCancel.html. Cited in v0.2; not re-checked for v1.0.
4. CSC2131 RDBMS Lab handout, provided by the team (Labs 11 and 12).
5. CSC2104 official course handout: not yet obtained.

6. Earlier 14-page Railway SRS (21 October 2026): not re-read for v1.0. Its invariants and query list were carried forward through v0.2 and the team brief. Reconcile before sign-off.

## 15. Open items and sign-off

- [ ] Obtain the official CSC2104 handout; confirm or relax the Java style default.
- [ ] Confirm MySQL 8 is accepted for the assessed RDBMS portion.
- [ ] Confirm the exact deadline, submission format and what faculty expect to see (report, code, demo).
- [ ] Confirm team size and assign owners in 12.1.
- [ ] Complete the rule review in Appendix C with two reviewers before setting verified = true.
- [ ] Reconcile with the earlier 14-page SRS.
- [ ] Confirm Python 3 or shell is available for the tooling scripts.
- [ ] Approve this revision as the baseline; later changes need a dated decision entry.

## 16. Future scope

Nothing in this section is part of this submission. Each item says why it is deferred and what must be true before it is worth building.

### 16.1 Reservation engine

| Item | Why deferred | Prerequisite |
|---|---|---|
| JDBC adapter, local HTTP API, small frontend (Tier 3) | Not assessed; would compete with the two cores for time | Tier 1 and Tier 2 gates passed with evidence; the adapter calls the existing procedures instead of reimplementing the rules |
| java.util.concurrent variant of the queue and agents | The handout may restrict it; the fixed-array monitor version is the safe default | Official CSC2104 handout confirmed; R-02 and R-13 reused to compare behaviour |
| RAC, quotas, Tatkal, multi-leg journeys, fare charts | Each adds state-machine and fare rules beyond the lab syllabus | A written rule source for each, and new fixtures before code |
| Deadlock retry policy with measured contention | Demo scale is too small to measure | A larger synthetic dataset and a recorded baseline |
| Optimistic-locking comparison | Interesting but not needed to show correctness | Pessimistic version stable and fully tested |

### 16.2 Refund advisor

| Item | Why deferred | Prerequisite |
|---|---|---|
| Linked missed-connection refund | Needs linked-PNR modelling and a separately reviewed rule scope; excluded for IRCTC-operated trains | Two-person review of the general rules row (linked PNRs, TDR within three hours of the actual arrival of the train travelled); new fixtures; no invented connection-gap threshold |
| Concurrent-safe rule publication | Removed from 1.0 to protect the schedule; overlap is rejected by a trigger instead | Reintroduce REFUND_RULE_FAMILY with a family-row lock, a publish procedure and a two-session test |
| Partial cancellation, diversion, short termination, AC failure (20 hours per both IRCTC documents), coach problems | Outside the two mandatory families | Each needs its own sourced rule, deadline basis and fixtures |
| Waitlisted and RAC tickets, party and family PNRs | The official late-running clause covers some of them; excluded by project scope | Ticket-type model extended and refund treatment sourced per type |
| Refund components (fees, payment charges) | Only the base fare is modelled | A source for what is and is not refunded, then amounts split in the evaluation |
| Rule drift watch | Rules may be amended after the saved PDFs | A script that compares dated copies of the source PDFs and flags a change for human review |

### 16.3 Real-world use

Adding features does not turn the simulator into a passenger tool. Before any real-data step:

- Interview people who have actually filed TDRs, and compare with what the official channels and existing third-party apps already do. The value of deadline-aware guidance is a hypothesis until then.
- Use only data a passenger enters themselves. Never collect IRCTC credentials. Read running status only from sources whose terms permit it, and record the source and time of every reading.
- Keep outputs labelled as guidance, not a decision, and never file claims on a user's behalf.

- Treat UNRESOLVED as a first-class result in any user interface.

### 16.4 Engineering quality

- Property-based and mutation testing of the evaluator and the state machine.
- Continuous integration running tools/run_all on every commit, and a containerised MySQL for one-command setup.
- Structured logs for evaluations and race runs, with run counts and seeds kept as data.

### 16.5 Order of work if the project continues

1. Reintroduce concurrent-safe rule publication.
2. Add one new rule family with its own sourced fixtures, to test that the rules-as-data design holds.
3. Run the interviews in 16.3.
4. Only then consider Tier 3 or the linked-connection rule.

## Appendix A. Complete fixture example (D-01)

Rule line fields: version, effective_from, effective_to (blank = open), threshold minutes, deadline basis, refund basis, verified. Event line fields: type, event_time, recorded_at, source, value.

```text
id=D-01
family=LATE_OVER_3_HOURS_E_TICKET
rule=1|2026-01-01T00:00:00+05:30||180|ACTUAL_DEPARTURE|FULL_FARE|true
fare=1250.00
booking=CONFIRMED|SIMULATED_E_TICKET
policy_applicability=2026-11-03T08:00:00+05:30
evaluation=2026-11-03T12:00:00+05:30
event=SCHEDULED_DEPARTURE|2026-11-03T08:00:00+05:30|2026-11-03T07:00:00+05:30|SIMU
event=DELAY_AT_BOARDING_MINUTES|2026-11-03T08:00:00+05:30|2026-11-03T10:45:00+05:3
event=PASSENGER_NOT_TRAVELLED|2026-11-03T11:00:00+05:30|2026-11-03T11:00:00+05:30|
event=FILING_ATTEMPT|2026-11-03T11:15:00+05:30|2026-11-03T11:15:00+05:30|SIMULATED
event=ACTUAL_DEPARTURE|2026-11-03T11:30:00+05:30|2026-11-03T11:30:00+05:30|SIMULAT
expect.outcome=ELIGIBLE
expect.reasons=DELAY_OVER_THRESHOLD,FILED_BEFORE_DEPARTURE
expect.warnings=PASSENGER_STATUS_SELF_REPORTED
expect.deadline_at=2026-11-03T11:30:00+05:30
expect.action=MODELLED_TDR_FILED_BEFORE_DEPARTURE
expect.amount=1250.00
```

Check: 08:00 scheduled plus 210 minutes is 11:30, which matches the actual departure, and every event used was recorded before the 12:00 evaluation time.

## Appendix B. RDBMS query and lab checklist

| Topic | Required demonstration |
|---|---|
| SELECT, WHERE, ORDER BY, LIMIT | Confirmed bookings, bookings by class and date, most recent bookings |
| Operators | Fare ranges, class filters, status combinations, LIKE, NULL checks |
| Aggregates, GROUP BY, HAVING | Occupancy, confirmed and waitlisted counts, classes over a count threshold |
| Joins | Inner (booking, passenger, journey); outer (waitlisted with optional allocation); self (rule version history); cross (class template); natural only as a labelled teaching example |
| Subqueries | Scalar fare or threshold; IN list of full classes; correlated check for passenger-journey conflicts |
| Functions, COALESCE | occupancy_pct with zero seats; latest evaluation with no rows |
| Views | Waitlist order, latest evaluation per case, one updatable view tested |
| Privileges | Reader versus writer accounts; GRANT, REVOKE, SHOW GRANTS |
| TCL | COMMIT, ROLLBACK, SAVEPOINT with the state shown after each |
| Procedures, triggers, cursor | Book, cancel, promote, evaluate; audit and transition triggers; cursor over the waitlist and the report refresh |
| Dialect note | MySQL differs from Oracle PL/SQL; document equivalents; no Oracle-only MERGE or INSERT ALL |

## Appendix C. Rule review checklist

For each seed rule, two team members complete these steps before verified is set to true:

☐ Open the cited PDF and record the URL, title, clause and the date checked; save a dated copy under docs/sources/.
☐ Confirm the clause numbers cited in this SRS (5(a) and 6 in the general rules; the late-running clause in the IRCTC-trains rules, cited as 7) against the retrieved text.
☐ Record the exact conditions: the three-hour threshold and strictness, the passenger-not-travelled condition, the filing cutoff and the automatic-refund wording for cancelled trains.
☐ Record the assumptions: delay measured at the boarding station, confirmed simulated e-ticket only, base fare only.

- [ ] Note any amendment or newer version of the document since the copy was saved.
- [ ] Second reviewer repeats the check independently and initials it.
- [ ] Only then set verified = true, with reviewer and checked date filled.